République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Djilali Liabbès – Faculté des Sciences Exactes
Département d'Informatique

TRAVAUX PRATIQUES

Matière : Sécurité et Cryptographie Appliquée

TP N°1 : Introduction, Objectifs et Historique de la Cryptographie

Algorithme d'Euclide étendu et Inversion modulaire

Nom et Prénom : RAHAL Imene
Groupe : Groupe 1
Spécialité : Ingénierie des Systèmes Informatiques (ISI)
Année universitaire : 2025/2026
Date : 23 février 2026

Table des matières

1 Introduction **2**

2 Rappels théoriques **2**

2.1 Plus Grand Commun Diviseur (PGCD) 2

2.2 Identité de Bézout 2

2.3 Algorithme d'Euclide étendu 3

2.4 Inverse modulaire 3

2.5 Chiffrement affine 3

3 Code Python **4**

3.1 Classe `BigIntMath` 4

3.2 Classe `AffineCipher` 4

3.3 Exemple d'utilisation 6

4 Interprétation du programme **6**

1 Introduction

Le chiffrement affine est une méthode classique de cryptographie qui permet de transformer un message clair en message chiffré en utilisant des opérations arithmétiques simples. Le principe repose sur l'utilisation de deux clés, a et b, qui permettent de combiner une multiplication et une addition modulo la taille de l'alphabet pour obtenir le texte chiffré.

Les étapes principales pour réaliser un chiffrement affine sont les suivantes :

1. Choisir un alphabet de taille n (par exemple $n = 26$ pour les lettres majuscules de A à Z).
2. Sélectionner une clé a telle que $\gcd(a, n) = 1$ afin de garantir l'existence de l'inverse modulaire et donc la réversibilité du chiffrement.
3. Choisir une clé b pour le décalage.
4. Convertir chaque lettre du message clair en un entier m selon sa position dans l'alphabet.
5. Appliquer la fonction de chiffrement $E_{a,b}(m) = (a \cdot m + b) \bmod n$ pour obtenir le chiffre correspondant.
6. Transformer les nombres chiffrés en lettres pour former le message chiffré.
7. Pour le déchiffrement, calculer l'inverse modulaire de a modulo n et appliquer $D_{a,b}(c) = a^{-1} \cdot (c - b) \bmod n$ pour retrouver le message original.

Cette approche permet de comprendre de manière concrète comment l'arithmétique modulaire est utilisée pour sécuriser un message, et constitue une première étape vers la compréhension des systèmes de chiffrement plus complexes.

2 Rappels théoriques

2.1 Plus Grand Commun Diviseur (PGCD)

Soient $a, b \in \mathbb{Z}$, non tous deux nuls. Le **plus grand commun diviseur** de a et b, noté $\gcd(a, b)$, est le plus grand entier positif divisant simultanément a et b.

2.2 Identité de Bézout

Il existe toujours des entiers relatifs u et v tels que :

$$au + bv = \gcd(a, b).$$

for inverse
$au + bv = 1$
s.t $\gcd(a,b) = 1$

Cette relation est appelée **identité de Bézout**.

2.3 Algorithme d'Euclide étendu

L'algorithme d'Euclide étendu calcule non seulement le PGCD de deux entiers, mais aussi les coefficients de Bézout u et v. Le principe est de maintenir, à chaque itération, une représentation linéaire du reste courant en fonction des valeurs initiales.

Algorithm 1 Algorithme d'Euclide étendu

Require: $a, b \in \mathbb{Z}$, $b \neq 0$

Ensure: (r, u, v) tels que $r = \gcd(a, b)$ et $r = au + bv$

```
r ← a, r' ← b
u ← 1, v ← 0
u' ← 0, v' ← 1
while r' ≠ 0 do
    q ← r ÷ r'                                   ▷ division entière
    (r, u, v, r', u', v') ← (r', u', v', r − qr', u − qu', v − qv')
end while
return (r, u, v)
```

2.4 Inverse modulaire

Soient $a, n \in \mathbb{Z}$ avec $n \geq 2$. On dit que a est **inversible modulo** n s'il existe $x \in \mathbb{Z}$ tel que :

$$ax \equiv 1 \pmod{n}.$$

L'inverse modulaire de a modulo n existe si et seulement si $\gcd(a, n) = 1$. Dans ce cas, l'algorithme d'Euclide étendu appliqué à (a, n) fournit :

$$au + nv = 1,$$

et la valeur $u \bmod n$ (comprise entre 0 et $n - 1$) est l'inverse modulaire recherché.

2.5 Chiffrement affine

Soit $n \geq 2$ la taille de l'alphabet (par exemple $n = 26$ pour l'alphabet latin majuscule). Le **chiffrement affine** est défini par :

$$E_{a,b}(m) = (a \cdot m + b) \bmod n,$$

avec la condition $\gcd(a, n) = 1$ pour garantir l'injectivité (donc la réversibilité). Le déchiffrement utilise l'inverse modulaire de a :

$$D_{a,b}(c) = a^{-1} \cdot (c - b) \bmod n,$$

S.t $K = (a, b)$

où a^{-1} est l'inverse de a modulo n.

3 Code Python

3.1 Classe BigIntMath

```python
class BigIntMath:
    @staticmethod
    def extendedGCD(a: int, b: int):

        r, r_prime = a, b
        u, v = 1, 0
        u_prime, v_prime = 0, 1

        while r_prime != 0:
            q = r // r_prime
            r, r_prime = r_prime, r - q * r_prime
            u, u_prime = u_prime, u - q * u_prime
            v, v_prime = v_prime, v - q * v_prime

        return r, u, v

    @staticmethod
    def modInverse(a: int, n: int):
        """
        Calcule l'inverse modulaire de a modulo n.
        Retourne None si l'inverse n'existe pas.
        """
        d, u, _ = BigIntMath.extendedGCD(a, n)
        if d != 1:
            return None  # pas d'inverse
        return u % n  # inverse modulaire dans [0, n-1]
```

Listing 1 – Algorithme d'Euclide étendu et inverse modulaire

3.2 Classe AffineCipher

```python
class AffineCipher:
    def __init__(self, a: int, b: int, n: int = 26):

        if BigIntMath.modInverse(a, n) is None:
            raise ValueError(f"{a} n'est pas inversible modulo {n}"
                )
```

```python
        self.a = a
        self.b = b
        self.n = n
        self.a_inv = BigIntMath.modInverse(a, n)

    def _char_to_int(self, char):
        """Convertit une lettre majuscule en entier (A=0, B=1, ..., Z=25)"""
        return ord(char) - ord('A')

    def _int_to_char(self, num):
        """Convertit un entier en lettre majuscule"""
        return chr(num + ord('A'))

    def encrypt(self, plaintext: str) -> str:

        plaintext = plaintext.upper()
        ciphertext = ""
        for c in plaintext:
            if c.isalpha():  # seulement les lettres
                m = self._char_to_int(c)
                ciph = (self.a * m + self.b) % self.n
                ciphertext += self._int_to_char(ciph)
            else:
                ciphertext += c
        return ciphertext

    def decrypt(self, ciphertext: str) -> str:

        ciphertext = ciphertext.upper()
        plaintext = ""
        for c in ciphertext:
            if c.isalpha():
                c_num = self._char_to_int(c)
                m = (self.a_inv * (c_num - self.b)) % self.n
                plaintext += self._int_to_char(m)
            else:
                plaintext += c
        return plaintext
```

Listing 2 – Chiffrement et déchiffrement affine

3.3 Exemple d'utilisation

```python
a = 5
b = 8
message = "HELLO WORLD"

cipher = AffineCipher(a, b)

encrypted = cipher.encrypt(message)
decrypted = cipher.decrypt(encrypted)

print("Message clair :", message)
print("Chiffre       :", encrypted)
print("Dechiffre     :", decrypted)
```

Listing 3 – Exemple de chiffrement et déchiffrement

L'exécution de ce code donne :

```
Message clair : HELLO WORLD
Chiffré       : RCLLA OWGDL
Déchiffré     : HELLO WORLD
```

Ce qui vérifie expérimentalement que $D_{a,b}(E_{a,b}(m)) = m$.

4 Interprétation du programme

- La classe `BigIntMath` fournit l'algorithme d'Euclide étendu, qui retourne le PGCD et les coefficients de Bézout. Cette implémentation suit rigoureusement le pseudo-code donné dans les rappels théoriques.
- La méthode `modInverse` utilise ces coefficients pour calculer l'inverse modulaire lorsque $\gcd(a, n) = 1$, et signale l'absence d'inverse dans le cas contraire.
- La classe `AffineCipher` exploite cet inverse pour assurer le déchiffrement. Elle vérifie dès l'initialisation que la clé a est inversible modulo n, conformément à la condition mathématique.
- Les méthodes `encrypt` et `decrypt` opèrent sur des chaînes de caractères, en ignorant les caractères non alphabétiques pour plus de souplesse.
- L'exemple concret montre la parfaite réversibilité du chiffrement, validant ainsi l'implémentation.

1/1